How `search.py` works

Walkthrough · 12 October 2026 · `search_algorithm/final_algorithm/` · the search selected by the experiments: P3 + E1 + D3

SUMMARY

`search_algorithm/final_algorithm/search.py` turns the search the experiments selected into one module the Flask service can import. A process builds a `SearchIndex` once, then answers each query with `index.search(query)`, which returns person rows, best first. Until the database exists, the rows come from the transcription. On the experiments' own archive and test queries, the module returns exactly the answers the experiments measured.

1 Using it

```
from search import load_proxy_index

index = load_proxy_index()                  # once, at start-up: 0.2 s with the vector cache
index.search("eva bendersky", limit=3)      # per query: about 30 ms
```

```
cd search_algorithm/final_algorithm
python search.py --download-weights --build-vectors   # once: weights, then vectors
python search.py "eva bendersky"                       # search from the terminal
```

person_id	Latin names	Hebrew names	section · row · stone	died	score
689	EVA S. DE BENDERSKY	חוה בענדערסקי	women · 40 · 4	1959	**2.00**
292	Ana de Brunstein	העניה ברונשטיין	women · 6 · 10	1935	**1.29**
500	Sara Jarast de Brunstein	—	women · 18 · 6	1940	**1.29**

Table 1. `index.search("eva bendersky", limit=3)`. Each result is a person row plus its score: 2 when both parts of the name match exactly, 1 for one part, 0.9 for one part found through the dictionary. People scoring 0 are left out.

2 Where the data comes from

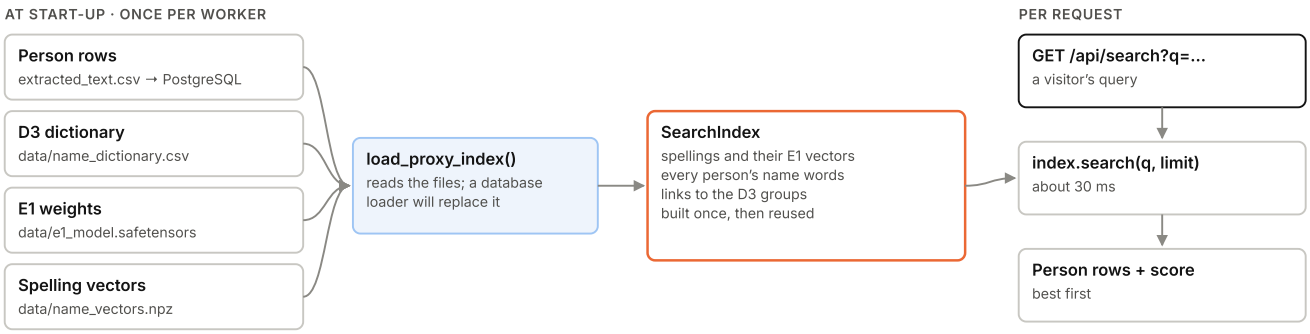


Figure 1. The index receives its data once, when the service starts, and every request reuses it. Only the loader knows where the rows come from.

The index is given its data rather than importing it, and is given it once rather than with every call:

- **Not with every call.** Building the index is the costly part: a vector for every spelling, the dictionary links and the token arrays. Passing data with each search would redo that work, or push the bookkeeping into every route.
- **Not at import.** A module that reads files when it is imported is tied to their paths, slows every import, cannot be tested without real data (as the Code Style requires), and cannot reload after an admin edits a record.
- **Once, at start-up.** gunicorn keeps each worker alive across requests, so the Flask app builds one index when it starts. Moving to PostgreSQL means writing a loader that reads the same rows from the database; `SearchIndex` does not change.

File	Stands in for	Made by	In git
`ocr_pipeline/extracted_text.csv`	`person` rows; a row's position is its `person_id`	OCR pipeline	yes
`data/name_dictionary.csv`	D3, as a static file (or a future table)	experiments, step 3	yes
`data/e1_model.safetensors`	E1's weights (MIT licence)	`--download-weights`	no
`data/name_vectors.npz`	the `name_vector` table that `import.py` will fill	`--build-vectors` (≈ 40 s)	no

Table 2. The proxy data; `data/` is the folder next to `search.py`. The download is pinned to the commit the experiments used and checked against their SHA-256.

3 How a search runs

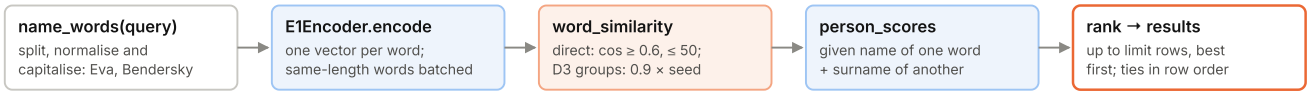


Figure 2. The functions a query passes through. E1 encodes only the query's words; the archive's vectors were computed when the index was built.

Building the index does all the work that does not depend on the query: `collect_names` gives every spelling its display form, and `link_spellings` links every archive spelling to the D3 groups of its own spelling or of the dictionary spelling within cosine 0.95. E1 reads raw bytes and is case-sensitive ("rebeca" and "Rebeca" score 0.86), so `display_spelling` capitalises archive and query words alike, and a visitor may type in any case.

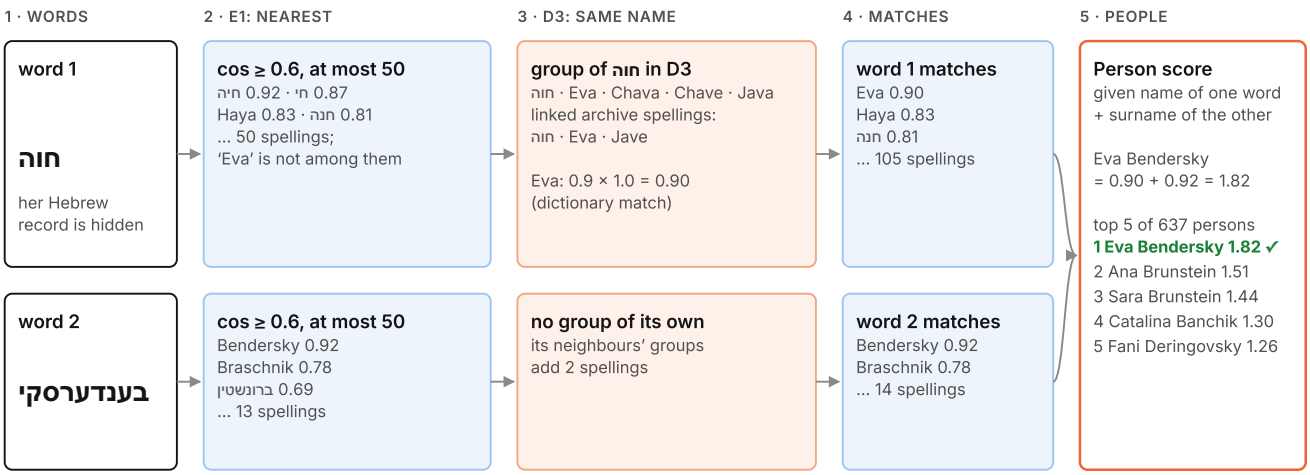


Figure 3. One query through `word_similarity` (columns 2–4) and `person_scores` (column 5), from the experiments' test set, with the person's Hebrew record hidden.

4 Module map

Section	Contents	Purpose
SETTINGS	`CUTOFF, MATCH_CAP, TAU_LINK, DICTIONARY_WEIGHT, DEFAULT_LIMIT, file paths`	the values the experiments tuned
NAMES	`word_script, normalise, display_spelling, name_words`	turn a name field or a query into words; to be shared with `import.py`
E1	`E1Encoder.from_file, E1Encoder.encode`	the name encoder in NumPy, words of equal length in one batch
INDEX	`collect_names, link_spellings, SearchIndex: search, rank, word_similarity, person_scores, spelling_vectors`	build once; search per query
PROXY DATA	`person_row, load_vectors, save_vectors, download_e1_weights, load_proxy_index`	stand in for PostgreSQL
CLI	`main`	search from the terminal; prepare the data files

Table 3. The module's six sections, in file order (469 lines).

5 Checks

- **Parity with the experiments.** On the experiments' 637 persons and 491 test queries, the module returns the same first five persons as step 8 for all 491 queries; R@5 is 0.885, as measured there. Cross-script queries are run with the target's own-script record hidden, as in the experiments.
- **Unit tests.** 16 tests on synthetic data (hand-made vectors and small random E1 weights), kept outside the repository. They cover normalisation, batched encoding, the full-name rule, the dictionary weight, links, the cut-off, limits, the vector cache and the download check.
- **Pylint.** 10/10 under Python 3.14.
- **Speed, on a busy laptop.** The index loads from the cache in 0.2 s. A search takes 27 ms at the median and 93 ms at the 95th percentile, nearly all of it E1's encoding of the query; matching and scoring take under 1 ms.

6 Before deployment

- **Design update.** The Project Design specifies Daitch–Mokotoff codes for search. This module searches with E1 and D3, which needs E1's vectors (a `name_vector` table, or a file) and the dictionary (a file shipped with the module). This has to be agreed and written into Notion first.
- **Database loader.** Replace `load_proxy_index` with a loader that reads `person` rows and their vectors from PostgreSQL. `import.py` computes the vectors with `E1Encoder` and reuses `name_words`.
- **Render build.** Run `--download-weights --build-vectors` in the build step, or read the vectors from the database, then time a search on the Render instance.
- **Not yet covered.** Maiden, father's and husband's names (the experiments searched given names and surnames only), filters, and a minimum score for showing a result.